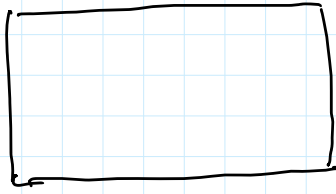


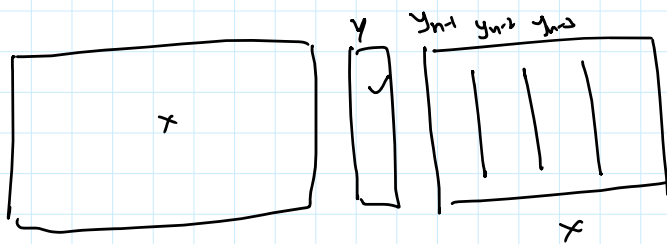
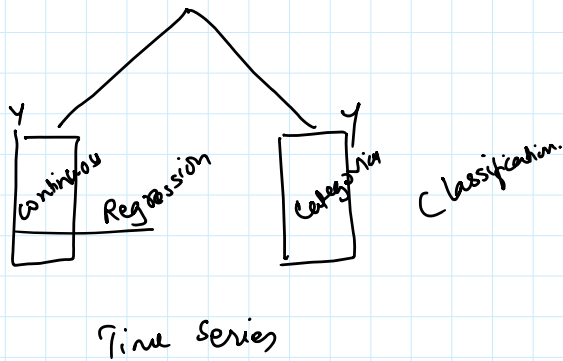
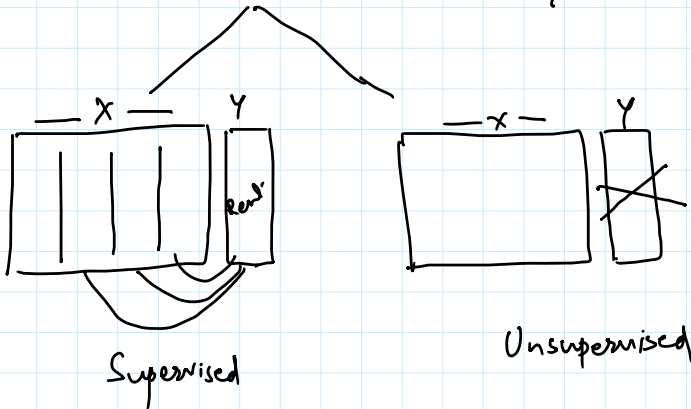
Structured Data



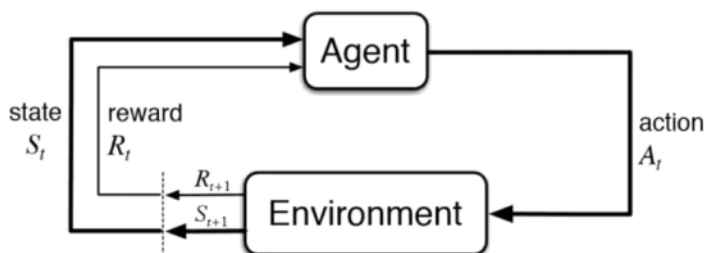
Unstructured Data

Images, Text,
Audio, Video.

↓
Deep learning

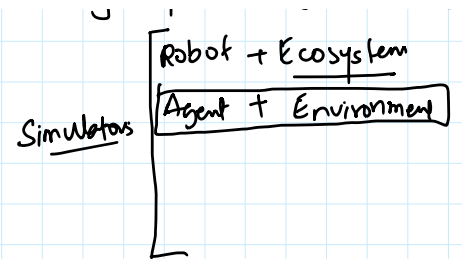
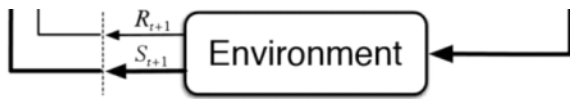


Reinforcement Learning



$R+1$
 $R-1$ } 10mth

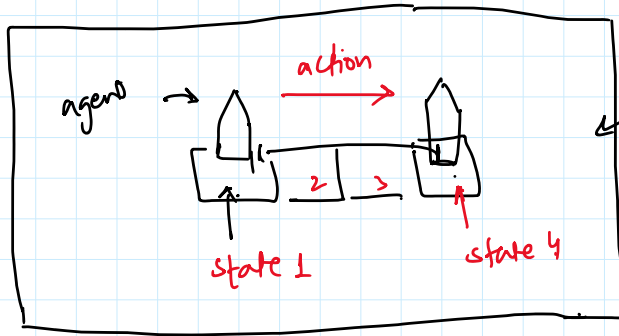
Library → Gymnasium [Deep Mind] AWS
Robot + Ecosystem
[A. B. + E. ...]



Reward $\rightarrow 0$ Reward $\rightarrow +100$
 Reward $\rightarrow +1$ $\swarrow$ Go

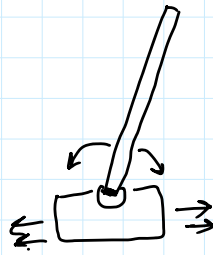
Reward $\rightarrow -1ve$

Game Over $\rightarrow 0$



Gymnasium
 $\downarrow$
 frozen lake.
 Cart Pole

Cart-Pole



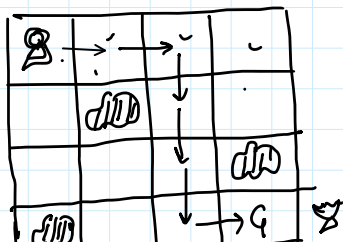
Agent Space
 Environment Space

θ movement $\rightarrow$ degree
 $\dot{\theta}$ movement $\rightarrow$ speed
 x movement $\rightarrow$ magnitude
 $\dot{x}$ movement $\rightarrow$ speed

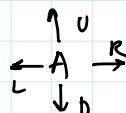
Agent Space $\rightarrow$ Action Space
 Environment Space $\rightarrow$ Observation Space.

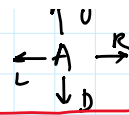
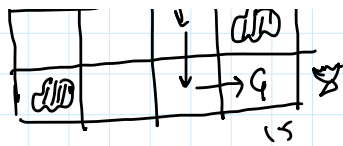
Cart - Pole.
 frozen lake.

Action $\rightarrow$ (Discrete)
 Observation $\rightarrow$ (Discrete)

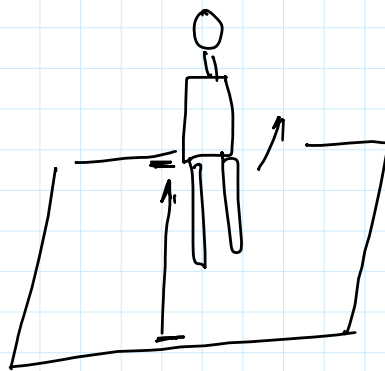
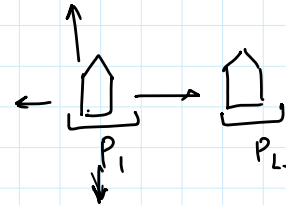
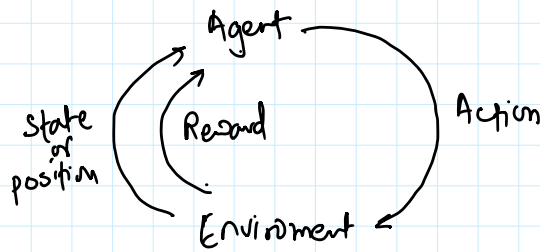


$G \rightarrow$ Max Reward





Action space $\rightarrow [L, D, R, U]$
 $\# 4$
 Observation space $\rightarrow [0, 1, 2, 3, 4, \dots, 15]$
 $\# 16$



Action space [Walk, Stop]

Falls $\rightarrow -R \rightarrow \begin{matrix} -10 \\ -5 \\ -20 \end{matrix}$

Not falling $\rightarrow +R$

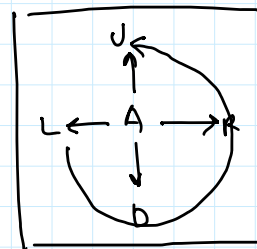
Environment $\rightarrow$ Big Floor

Library $\rightarrow$ OpenAI (Gym)
 $\rightarrow$ Gymnasium

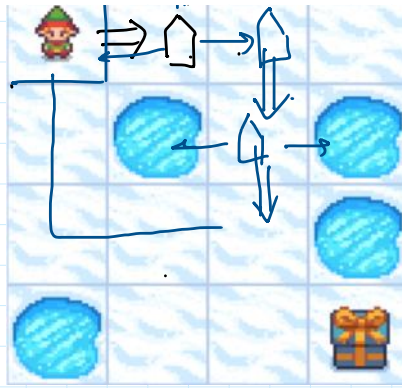
Action Space
 Observation Space



$A \rightarrow 0, 1, 2, 3$
 $\downarrow \downarrow \downarrow \downarrow$
 $L \quad D \quad R \quad U$



$\begin{bmatrix} R \\ U \end{bmatrix} \rightarrow$ Route



$\begin{matrix} R \\ U \\ D \\ D \\ D \\ R \end{matrix}$
 $\rightarrow$ Route
(Policy)

Deterministic Environment

- Same action in the same state always gives the same next state.
- Example: Chess (ignoring opponent randomness).

Stochastic Environment

- Same action can lead to different outcomes.
- Example: Robot navigation with sensor noise.

For a deterministic environment:

$$P(s' | s, a) = 1$$

for exactly one next state.

For a stochastic environment:

$$P(s'_1 | s, a) = 0.8$$

$$P(s'_2 | s, a) = 0.2$$

Fully Observable

- Agent can see the complete state of the environment.
- Example: Chess board.

Partially Observable

- Agent sees only part of the state.
- Example: Poker (opponent's cards are hidden).

Episodic

- Interaction ends after some goal or failure.
- Example: A game of Tic-Tac-Toe.

Continuing

- No natural ending.
- Example: Stock trading system.

Static

- Environment does not change while the agent decides.
- Example: Crossword puzzle.

Dynamic

- Environment changes continuously.
- Example: Self-driving car in traffic.

Discrete

- Finite states and actions.
- Example:
 - Grid World
 - Frozen Lake

Continuous

- Infinite range of states/actions.
- Example:
 - Robotic arm angles
 - Drone control

Single-Agent

- One agent interacts with the environment.
- Example: CartPole.

Multi-Agent

- Multiple agents interact.
- Example:
 - Chess
 - Soccer simulation

Known Environment

- Transition probabilities are known.
- Example: Simulators where rules are provided.

Unknown Environment

- Agent must learn dynamics through interaction.
- Example: Real-world robotics.

Environment	Type
Multi-Armed Bandit	Stateless
GridWorld	Discrete
FrozenLake	Stochastic
Taxi-v3	Discrete
CartPole	Continuous state
MountainCar	Continuous state
LunarLander	Continuous state
Atari Games	High-dimensional
MuJoCo Robotics	Continuous control

Single-Agent

- One agent interacts with the environment.
- Example: CartPole.

Multi-Agent

- Multiple agents interact.
- Example:
 - Chess
 - Soccer simulation

- Thermostat
- Automatic door

In RL terms, this agent doesn't remember past states.

Maintains an internal model of the environment.

It keeps track of:

- Current state
- How the world changes
- Effects of actions

Example:

- Robot vacuum that remembers cleaned areas.

A self-driving car may prefer:

- Safe route (utility = 90)
- Fast route (utility = 80)

It chooses the safer route despite taking longer.

Improves through experience.

Components:

- Learning element
- Critic (feedback)
- Performance element
- Problem generator (exploration)

Most RL agents belong to this category.

Examples:

- AlphaGo
- DQN agents
- PPO agents
- Chatbots trained with RLHF

position $\rightarrow$ env.step(A) $\rightarrow$ Next Position
 $\boxed{2}$ $\downarrow$ $\boxed{6}$
 $I(0)$

```
for k in env.unwrapped.P[14][2]:  
    print(k)
```

```
(0.3333333333333333, 14, 0, False)  
(0.3333333333333333, 15, 1, True)  
(0.3333333333333333, 10, 0, False)
```

$[0, 1, 2, 3] \# 4$

Action $\left\{ \begin{array}{l} \text{Integers} \\ \text{Position} \end{array} \right.$

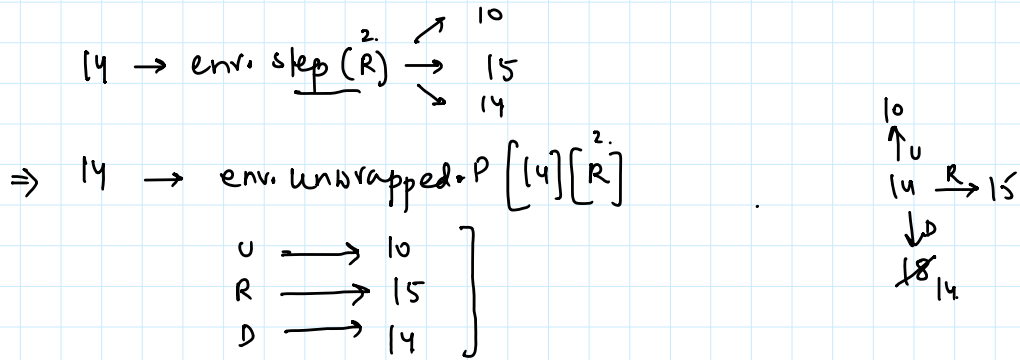
$[0, 1, 2, \dots, 19, 15] \# 16$

```

for a,b,c,d in env.unwrapped.P[14][2]:
    print(a,b,c,d)

→ 0.3333333333333337 14 0 False
→ 0.3333333333333333 15 1 True
→ 0.3333333333333337 10 0 False

```



```

for a,b,c,d in env.unwrapped.P[14][2]:
    print(a,b,c,d)

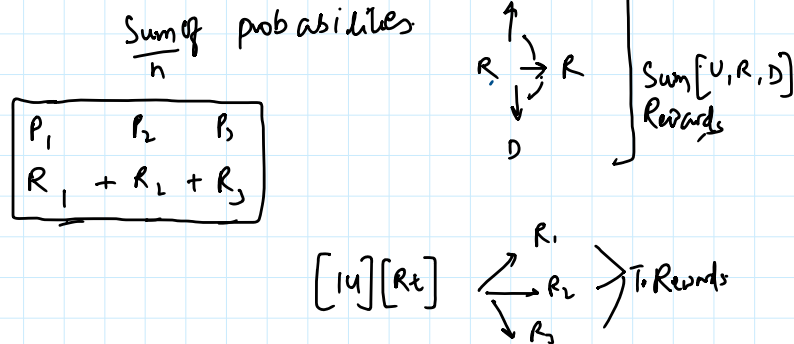
0.3333333333333337 14 0 False
0.3333333333333333 15 1 True
0.3333333333333337 10 0 False

```

Game Over

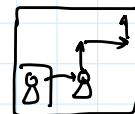
$$0 + 1 + 0 \Rightarrow 1$$

Best / Optimum route



γ
 γ^2
 γ^3

Monte Carlo



$$G = R_1 + R_2 + R_3 + R_4 + \dots$$

$$G = R_1 + \gamma R_2 + \gamma^2 R_3 + \gamma^3 R_4$$

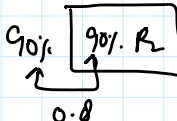
$\downarrow 0.9$ $\downarrow 0.81$

$\gamma \rightarrow$ Discount factors -

$$\gamma \rightarrow 0.9 \sim 1.0$$

$$\gamma^2 = 0.9^2 = 0.81$$

$\gamma \neq \boxed{x} \rightarrow 100\%$
 γ



full reward $\rightarrow G = R_0 + \gamma R_1 + \gamma^2 R_2 + \gamma^3 R_3 - \dots$

$$= R_0 + \gamma [R_1 + \gamma R_2 + \gamma^2 R_3 + \dots]$$

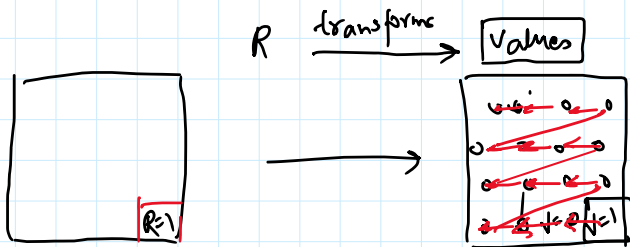
full return

$$= R_0 + \gamma [R_1 + \gamma R_2 + \gamma^2 R_3 + \dots]$$

$$G = R_0 + \gamma G_1$$

$$V_{max} = \max_p (R + \gamma V)$$

Bellman Optimality Eq.

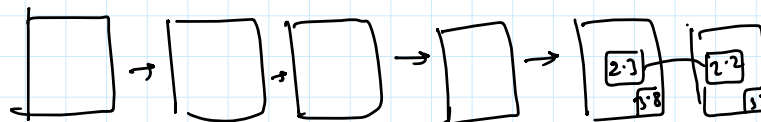


Model Based
(Full knowledge of environment)
[All the possibilities are known in advance]



$$V \rightarrow \frac{\text{Avg Reward}}{13} + \text{Grid}$$

$$\begin{matrix} \text{Prev} \\ \text{Round} \end{matrix} \left[\begin{matrix} V(S_{next}) \\ 14 \end{matrix} \right] + \begin{matrix} \text{Present} \\ \text{Round} \end{matrix} \left[\begin{matrix} R(S_{present}) \\ 13 \end{matrix} \right] \rightarrow \begin{matrix} \text{New Value} \\ 13 \end{matrix}$$



```

0, 1, 2, 3
for act in range(4):
    for prob, next_state, reward, term in env.unwrapped.P[14][act]:
        print(prob, next_state, reward, term)

```

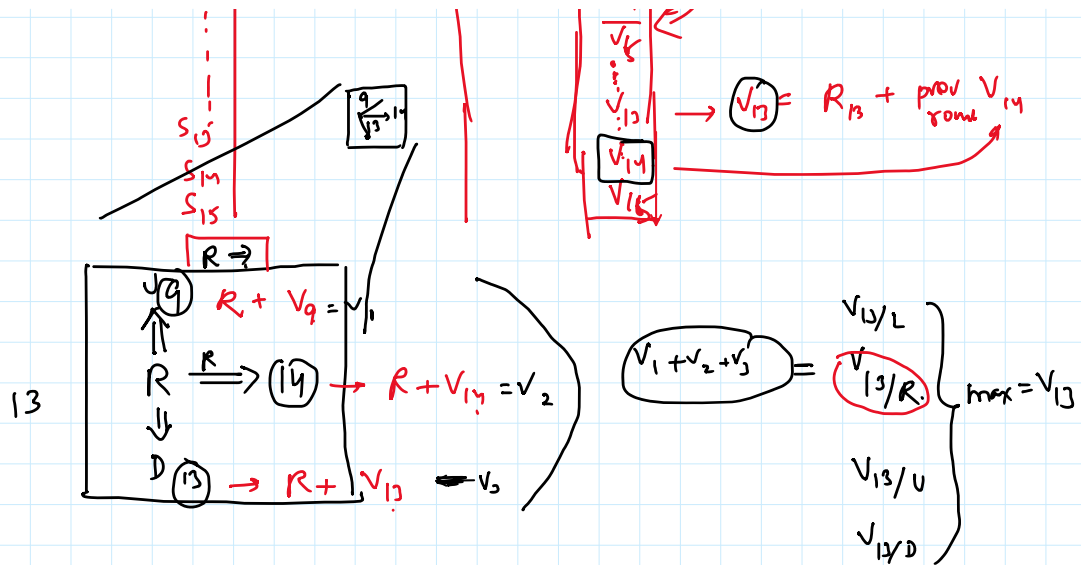
state
Action

0
1
2
3

```

for states in range(16):
    for action in range(4):
        for prob, next_state, reward, term in env.unwrapped.P[states][action]:
            print(prob, next_state, reward, term)

```

$$\begin{aligned}
 V_{14} &\rightarrow \left(\text{Reward } (15) + \gamma V_{next_state} \right) \\
 V_9 &\rightarrow \left(\text{Reward } (13) + \gamma V_{9_next_state} \right) \\
 V_{13} &\rightarrow \left(R_{(15)} + \gamma V_{13_next_state} \right)
 \end{aligned}$$

Sum of $\gamma V \rightarrow V_{13}/R$ } $V_{max} = V_{max,13}$

$$V_{max}[14] = 1$$

```

Vmax = [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
for k in range(20):
    for state in range(16):
        ls = []
        for action in range(4):
            value = 0
            for prob, next_state, reward, term in env.unwrapped.P[state][action]:
                value = value + prob*(reward + 0.9 * Vmax[next_state]) # Bellman Equation
            ls.append(value)
        Vmax[state] = max(ls)
    # print("-----")

```

